

国家级精品课程
山东省一流本科课程



程序设计基础（C语言）

信息科学与工程学院

潘玉奇

第3章 程序的控制结构

- ◆ 3.1 关系运算与逻辑运算
- ◆ 3.2 分支结构
- ◆ 3.3 循环结构
- ◆ 3.4 break和continue语句



3.1 关系运算与逻辑运算

3.1.1 关系运算

【问题描述3.1】 猜数字游戏，事先给出一个确定的数字(如123)，让游戏者从键盘输入他猜想的数字，如果游戏者猜对了，显示“Right”，猜错了显示“Wrong”。

分析：

问题的关键是将游戏者猜的数字与事先给出的数字进行比较。在计算机程序中进行这种比较需要将事件符号化。假设程序设定的数字为“123”，所猜的数字放入变量guess，判断猜测的数字是否正确，就是比较guess的值是否等于123，这就是所谓关系运算。

关系运算是对于两个表达式进行比较并产生运算结果。

3.1.1 关系运算

1. 关系运算符 $>$ $>=$ $<$ $<=$ $==$ $!=$

(1) 优先级：关系运算符优先级低于纯算术类运算符，高于赋值类运算符；前4种关系运算符优先级高于后2种

(2) 结合性：左结合性

注意：(1) 字符数据按其ASCII码值进行比较

(2) 应避免两个实数作 $==$ 或 $!=$ 的比较

2. 关系表达式：用关系运算符连接起来的式子

关系表达式的值只有2个，1表示真，0表示假

若关系表达式成立，其值为1（在C中非零值均表示逻辑真）

若关系表达式不成立，其值为0（零表示逻辑假）

注意： $5>2>7>8$ 是允许的，它相当于 $((5>2)>7)>8$

3.1.1 关系运算

例: `char c='w';     int i=1, j=2, k= -7;`
`double x=7e+03, y = 0.001;`

表达式	等价表达式	表达式的值
<code>'a'+1< c</code>	<code>('a'+1)< c</code>	1
<code>3<j<5</code>	<code>(3<j)<5</code>	1
<code>'8'-'0'==1</code>	<code>('8'-'0')==1</code>	0
<code>'1'=='0'+1</code>	<code>'1'==('0'+1)</code>	1
<code>x<x+y</code>	<code>x<(x+y)</code>	1
<code>i+j+k<=2*j</code>	<code>((i+j)+k)<=(2*j)</code>	1
<code>i<j==x<y</code>	<code>(i<j)==(x<y)</code>	0
<code>i=j==k<y</code>	<code>i=(j==(k<y))</code>	<code>i=0</code>
<code>i=k<y</code>	<code>i=(k<y)</code>	<code>i=1</code>

3.1.2 逻辑运算

1. 逻辑运算符 **!** **&&** **||**

(1) 3个运算符的优先级由高到低为: **! → && → ||**

(2) **!** 的优先级高于关系运算, **&&**和 **||** 的优先级低于关系运算

(3) **!** 是右结合性, **&&** 和 **||** 是左结合性

逻辑运算规则表

A	B	! A	A && B	A B
0	0	1	0	0
非0	0	0	0	1
0	非0	1	0	1
非0	非0	0	1	1

3.1.2 逻辑运算

2. 逻辑表达式

逻辑表达式的值应该是"逻辑真"或"逻辑假",
逻辑运算的结果以数值1代表"真",以数值0代表"假";
进行判断时,非零值代表"真",零值代表"假"

例: 表示 $x \in [-1, 1]$ 区间 应写为: $x \geq -1 \ \&\& \ x \leq 1$

3. 逻辑运算中的规律

在 $\&\&$ 表达式中,
若左端为0,则不必
计算右端,表达式
值为0

即: $0 \ \&\& \ a == 0$

规律:

$0 \ || \ a == a$

$1 \ || \ a == 1$

$a \ || \ a == a$

$a \ || \ !a == 1$

$!(a \ || \ b) == !a \ \&\& \ !b$

$0 \ \&\& \ a == 0$

$1 \ \&\& \ a == a$

$a \ \&\& \ a == a$

$a \ \&\& \ !a == 0$

$!(a \ \&\& \ b) == !a \ || \ !b$

3.1.2 逻辑运算

例: `char c = 'w';` `int i=j=k=3;` `double x=0.0, y = 2.3;`

表达式	等价表达式	表达式的值
<code>i && j && k</code>	<code>(i && j) && k</code>	1
<code>x && i j -2</code>	<code>(x && i) (j-2)</code>	1
<code>x i && j -3</code>	<code>x (i && (j-3))</code>	0
<code>i < j && x < y</code>	<code>(i < j) && (x < y)</code>	0
<code>i == j && x <= y</code>	<code>(i == j) && (x <= y)</code>	1
<code>x != y && j + 1 == !k + 4</code>	<code>(x != y) && ((j + 1) == (!k) + 4)</code>	1
<code>'A' <= c && c <= 'Z'</code>	<code>('A' <= c) && (c <= 'Z')</code>	0
<code>i = i == 2 j == 4 k == 6</code>	<code>i = ((i == 2) (j == 4) (k == 6))</code>	<code>i = 0</code>



3.1.2 逻辑运算

例 `#include <stdio.h>`

```
int main( )
```

```
{ int x, y, z ;
```

```
  x = y = z = 0 ;
```

```
  ++x || ++y && ++z ;  $\longrightarrow$   $((++x) \parallel ((++y) \&\& (++z)) )$ 
```

```
  printf(" %d , %d , %d \n", x, y, z);
```

```
  x = y = z = -1 ;
```

```
  ++x && ++y || ++z ;  $\longrightarrow$   $((++x) \&\& (++y)) \parallel (++z)$ 
```

```
  printf(" %d , %d , %d \n", x, y, z);
```

```
  return 0;
```

```
}
```

输出结果:

1 , 0 , 0

0 , -1 , 0

3.1.2 逻辑运算

【问题描述3.2】判断某一年是否为闰年。所谓闰年，是指符合下面两个条件之一：① 能被4整除，但不能被100整除；
② 能被4整除，又能被400整除。

分析：

- 对于条件①，能被4整除写作：`year%4==0`；不能被100整除写作：`year%100!=0`。要求两者同时满足，内部是一个“逻辑与”的关系，可合并为：`year%4==0&&year%100!=0`
- 对于条件②，因为能够被400整除一定能被4整除，所以第2个条件可以简化为能够被400整除，写作：`year%400==0`
- 条件①和条件②满足任何一个都是闰年，两者之间是“逻辑或”的关系。因此，判断闰年条件的逻辑表达式表示为：
`((year%4==0)&&(year%100!=0))||(year%400==0)`
- 表达式为“真”，即条件成立是闰年，否则不是闰年。

3.1.2 逻辑运算

【练习3.1】输入三条边长，判断这三条边是否能构成一个三角形。假设用三个变量是 x ， y ， z 存放三条边的边长，以下表达式哪一个是正确的？

A) $x+y>z$

B) $x+y>z, x+z>y, y+z>x$

C) $x+y>z \ \&\& \ x+z>y \ \&\& \ y+z>x$

D) $x+y>z \ || \ x+z>y \ || \ y+z>x$



3.2 分支结构

➤ 分支结构的引入

顺序结构只能解决一些简单的问题, 遇到需要进行条件判断的问题, 就必须采用分支结构来处理。

- 请输出2个数中的较大的数。

- 请编程求解分段函数:

$$y = \begin{cases} -1 & x < 0 \\ 0 & x = 0 \\ 1 & x > 0 \end{cases}$$

- 根据“体指数”判断某人的肥胖程度:

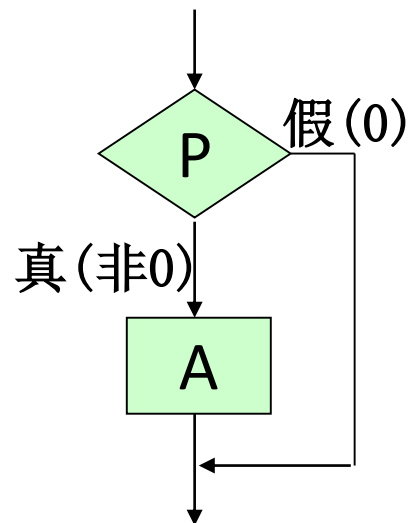
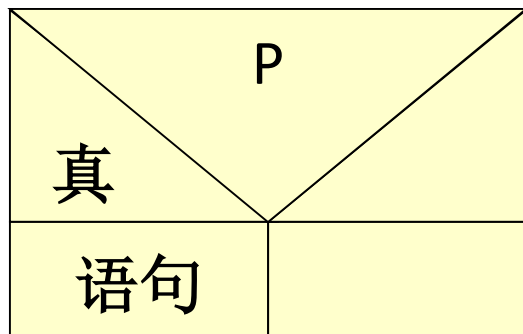
体指数 $t = w / h^2$, 其中: w —体重(千克), h —身高(米)

	中国标准
偏瘦	$\text{BMI} < 18.5$
正常	$18.5 \leq \text{BMI} < 24$
偏胖	$24 \leq \text{BMI} < 28$
肥胖	$\text{BMI} \geq 28$

3.2.1 单分支结构

1. 语法形式

if (表达式) 语句;



2. 执行过程

先计算表达式的值, 若为真(非0值), 则执行其后的语句, 若值为0, 则继续向下执行

3.2.1 单分支结构

【例3.1】 编写一个程序，判断某一年是否为闰年。

分析：

判断闰年的表达式已在【问题描述3.2】中给出，如果表达式值为真，则是闰年，输出“xxxx年是闰年”，否则不输出。

```
#include <stdio.h>
```

```
int main( )
```

```
{ int Year;
```

```
    printf("Input year:");
```

```
    scanf("%d", &Year);
```

```
    if (((Year%4==0)&&(Year%100!=0))||(Year%400==0))
```

```
        printf("%d年是闰年\n",Year);
```

```
    return 0;
```

```
}
```

3.2.1 单分支结构

【练习3.2】输入两个数，求最大数并输出，测试数据如下。

输入：12 35

输出：35

输入a , b	
真	假
a>b?	
输出a	
真	假
b>=a?	
输出b	

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int a, b;
```

```
    scanf("%d%d", &a, &b);
```

```
    if(a>b ) printf("%d\n", a);
```

```
    if(b>=a) printf("%d\n", b);
```

```
    return 0;
```

```
}
```


3.2.1 单分支结构

【练习3.4】输入三个整数，求最大数并输出。

程序1:

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int a, b, c;
```

```
    scanf("%d%d%d",&a,&b,&c);
```

```
    if(a>b>c) printf("%d\n", a);
```

```
    if(b>c>a) printf("%d\n", b);
```

```
    if(c>a>b) printf("%d\n", c);
```

```
    return 0;
```

```
}
```

错误：判断条件的表达式写法是错的，
输入不同数据，输出可能对，也可能
无输出结果

```
3 2 1
-----
1 2 3
-----
1 3 2
-----
3 2 0
3
-----
0 3 2
3
-----
Process exited after
请按任意键继续. . .
```

3.2.1 单分支结构

【练习3.4】输入三个整数，求最大数并输出

程序2:

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int a, b, c;
```

```
    scanf("%d%d%d",&a,&b,&c);
```

```
    if(a>b && a>c) printf("%d\n", a);
```

```
    if(b>a && b>c) printf("%d\n", b);
```

```
    if(c>a && c>b) printf("%d\n", c);
```

```
    return 0;
```

```
}
```



错误：当输入的3个数据互不不同时能得到正确答案，但当测试数据中有相同的数的时候没有输出

```
1 3 3
-----
3 1 3
-----
3 3 2
-----
3 3 3
-----
Process exited after
请按任意键继续. . .
```

3.2.1 单分支结构

【练习3.4】输入三个整数，求最大数并输出

程序3:

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int a, b, c;
```

```
    scanf("%d%d%d",&a,&b,&c);
```

```
    if(a>=b && a>=c) printf("%d\n", a);
```

```
    if(b>=a && b>=c) printf("%d\n", b);
```

```
    if(c>=a && c>=b) printf("%d\n", c);
```

```
    return 0;
```

```
}
```

错误：当测试数据中有相同的数时候
会重复输出相同的数

```
1 3 3
3
3
-----
3 1 3
3
3
-----
3 3 3
3
3
3
-----
Process exited after
请按任意键继续. . .
```



3.2.1 单分支结构

98456, 97981, 98934, 98971, 98765
98657, 97989, 97899, 98977, 97889
98776, 98487, 98734, 98653, 98855
97968, 98989, 98799, 98765, 98937
98695, 97873, 98689, 98948, 98975
97864, 98889, 97889, 98988, 98894

程序4:

```
#include<stdio.h>
int main()
{  int a, b, c, max;
    scanf("%d%d%d", &a, &b, &c);
    max=a;
    if(b>max) max=b;
    if(c>max) max=c;
    printf("%d\n", max);
    return 0;
}
```

这30个数中的最大数是多少?

你是怎么找到的?

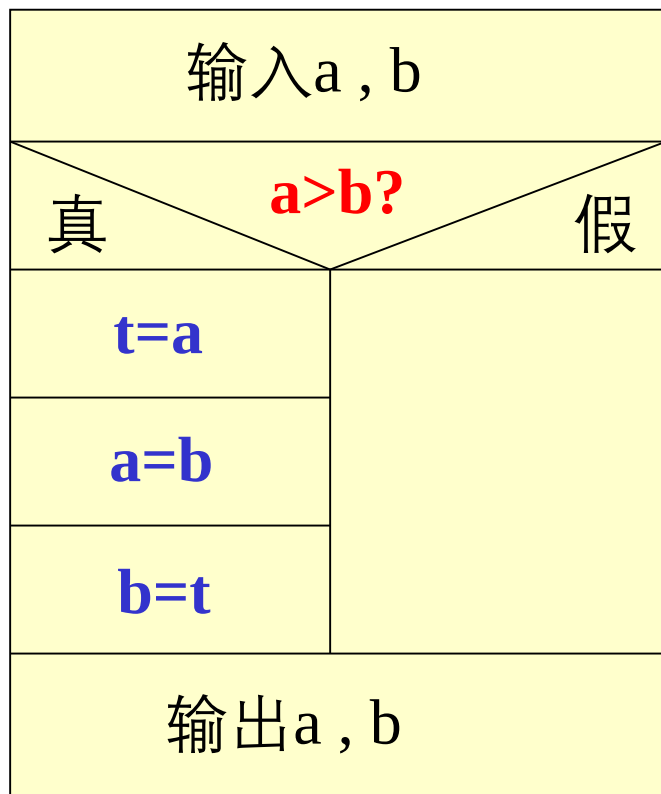
你能把该过程用代码描述出来吗?

这个方法能在30个数中找最大，肯定也能在3个数中找最大，其实这个方法是通用的，也可以在更多的数中找最大数。

程序4对于任何输入数据都能得到正确答案

3.2.1 单分支结构

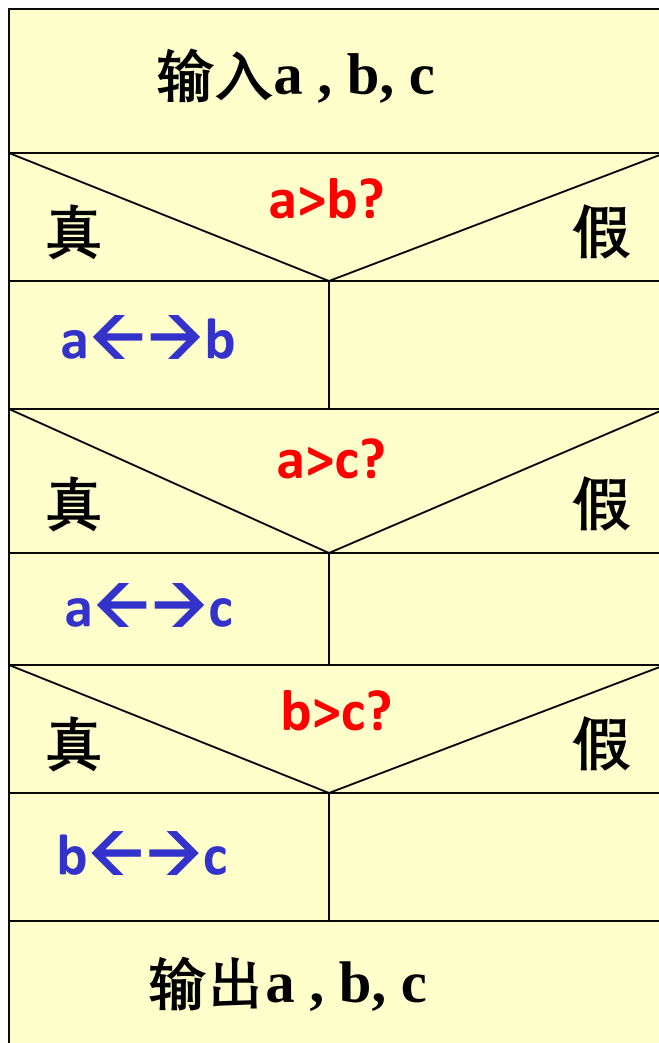
【例】 输入2个整数, 按从小到大的顺序输出



```
#include <stdio.h>
int main( )
{ int a, b, t;
  scanf("%d%d", &a, &b);
  if (a>b)
  { t=a;
    a=b;
    b=t;
  }
  printf("%d,%d\n", a, b);
  return 0;
}
```

3.2.1 单分支结构

【例3.3】输入3个整数，按从小到大的顺序输出这3个数。



```
#include <stdio.h>
```

```
int main( )
```

```
{ int a, b, c, t;
```

```
scanf("%d%d%d", &a, &b, &c);
```

```
if (a>b) { t=a; a=b; b=t; }
```

```
if (a>c) { t=a; a=c; c=t; }
```

```
if (b>c) { t=b; b=c; c=t; }
```

```
printf("%d %d %d\n", a, b, c);
```

```
return 0;
```

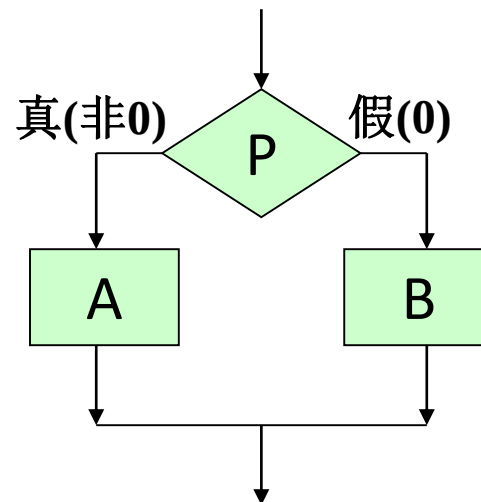
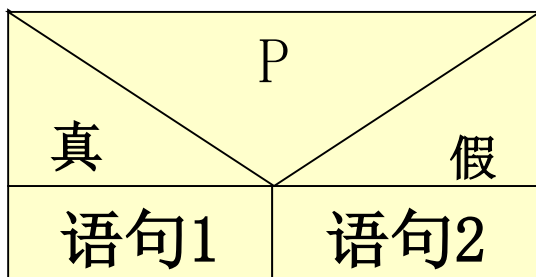
```
}
```

3.2.2 双分支结构

1. 语法形式

if (表达式) 语句1;

else 语句2;



2. 执行过程

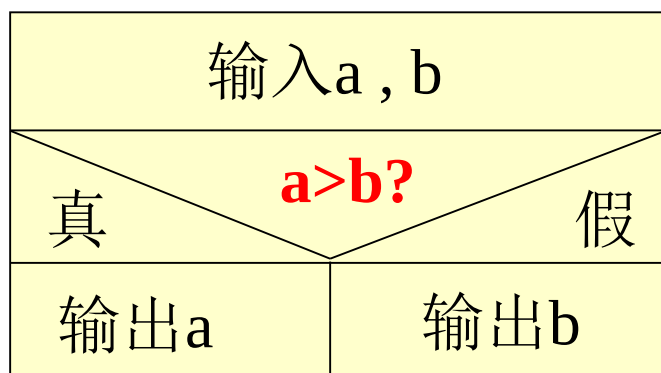
先计算表达式的值, 若为真(非0值), 执行语句1;
若值为0, 执行else后的语句2

3.2.2 双分支结构

【练习3.2】输入两个数，求最大数并输出，测试数据如下。

输入：12 35

输出：35



```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int a, b;
```

```
    scanf("%d%d", &a, &b);
```

```
    if( a>b ) printf("%d\n", a);
```

```
    else printf("%d\n", b);
```

```
    return 0;
```

```
}
```


3.2.2 双分支结构

【例】 编程求解函数

$$y = \begin{cases} 0 & x \leq 0 \\ x^3 + x + 1 & x > 0 \end{cases}$$

输入x	
真	假
$x \leq 0?$	
$y=0$	$y=x^3+x+1$
输出y	

```
#include <stdio.h>
#include <math.h>
int main( )
{ int x, y;
  scanf("%d", &x);
  if (x<=0) y=0;
  else y=pow(x, 3)+x+1;
  printf("y=%d\n", y);
  return 0;
}
```

3.2.2 双分支结构

【例3.4】编写一个密码判断程序，如果密码正确应给出正确提示，否则给出错误提示。(假设密码为110)

```
#include <stdio.h>
int main()
{
    int key;
    printf("请输入密码" );
    scanf ("%d", &key);
    if(key==110) printf("密码正确，欢迎使用！ \n");
    else printf("对不起，密码错误！ \n");
    return 0;
}
```



3.2.2 双分支结构

【练习3.3】 输入一个整数，判断它的奇偶性，是奇数输出**odd**，是偶数输出**even**。

测试数据1:	测试数据2:
输入: 23	输入: 32
输出: odd	输出: even

【练习】 输入一个整数，判断它能否同时被3、5、7整除，整除则输出**Yes**，否则输出**No**。

测试数据1:	测试数据2:
输入: 105	输入: 21
输出: Yes	输出: No

3.2.3 多分支结构

【问题描述3.3】 身体质量指数简称体指数BMI(Body Mass Index), 这是目前国际相对常用的一种衡量人体胖瘦程度以及健康状况的指标, 计算公式为: **体指数(BMI) = 体重(kg) ÷ 身高²(m)**
例如: 体重=55.8kg, 身高=1.70m, 则 $BMI = 55.8 \div (1.70)^2 = 19.31$

表3.4成人BMI指标

	中国标准
偏瘦	$BMI < 18.5$
正常	$18.5 \leq BMI < 24$
偏胖	$24 \leq BMI < 28$
肥胖	$BMI \geq 28$

如果让你来编程实现BMI的判断, 你会发现这里分了4种情况, 一个if...else...是不够用的, 所以需要用到多分支的选择结构。

3.2.3 多分支结构

1. if语句的多分支形式

if (表达式1)

语句1;

else if (表达式2)

语句2;

else if (表达式3)

语句3;

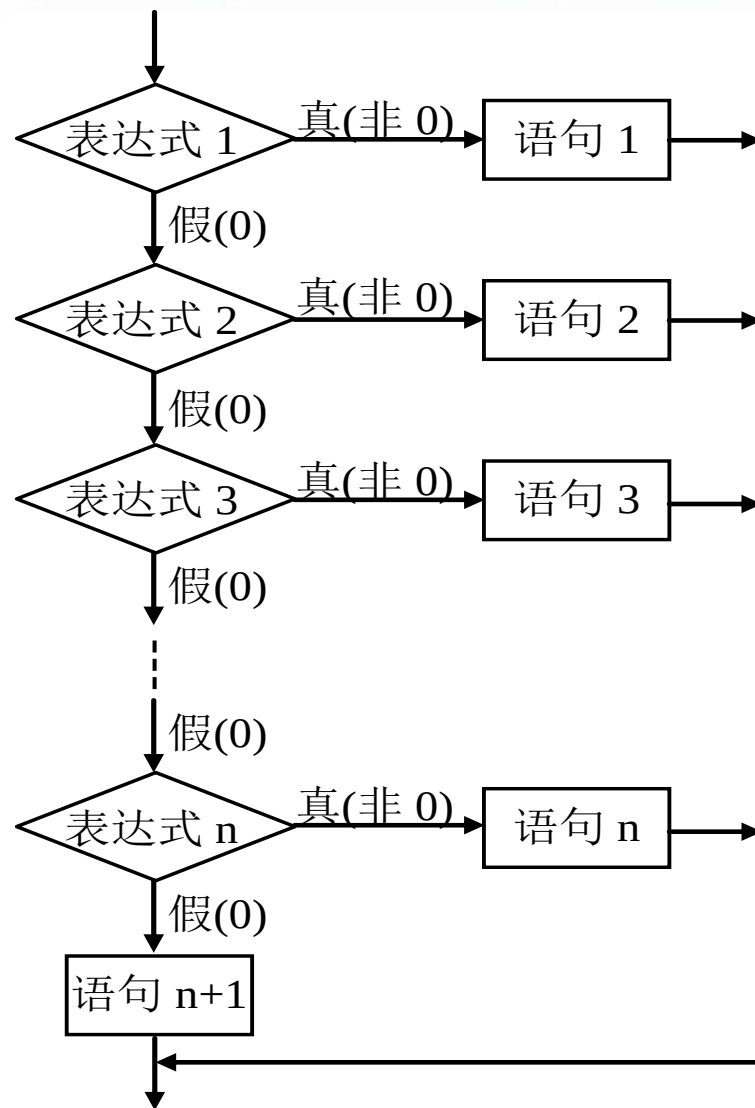
 ⋮

else if (表达式n)

语句n;

else

语句n+1;



3.2.3 多分支结构

【问题描述3.3】 计算体指数的参考程序

```
#include<stdio.h>
```

```
int main( )
```

```
{ double w, h, b; //w表示体重, h表示身高, b表示BMI
```

```
scanf("%lf%lf", &w, &h);
```

```
b=w/(h*h);
```

写成 $b=w/h*h$; 计算结果还对吗?

```
printf("BMI=%.2lf\n", b);
```

```
if (b<18.5) printf("偏瘦!\n");
```

```
else
```

写成 $\text{if}(b \geq 18.5 \ \&\& \ b < 24)$ 对吗?

```
if (b<24) printf("正常!\n");
```

```
else
```

```
if (b<28) printf("偏胖!\n");
```

```
else printf("肥胖!\n");
```

```
return 0;
```

else后为什么不写 $\text{if}(b \geq 28)$?

```
}
```

3.2.3 多分支结构

【例3.8】在学生成绩管理中，成绩经常要在百分制与等级制之间进行转换。90分以上为A等，80-89为B等，70-79为C等，60-69分为D等，其余为E等。编写程序，根据输入的百分制，输出对应的等级。

grade = {	A	$90 \leq s \leq 100$
	B	$80 \leq s < 90$
	C	$70 \leq s < 80$
	D	$60 \leq s < 70$
	E	$0 \leq s < 60$

3.2.3 多分支结构

【例3.8】百分制成绩转换为等级制成绩的参考程序1

```
#include<stdio.h>
```

```
int main( )
```

```
{ int score;
```

```
    printf("输入一个成绩: "); //可省略
```

```
    scanf("%d", &score);
```

```
    if(score>=90) printf("A\n");
```

```
    else
```

```
        if(score>=80) printf("B\n");
```

```
        else
```

```
            if(score>=70) printf("C\n");
```

```
            else
```

```
                if(score>=60) printf("D\n");
```

```
                else printf("E\n");
```

```
    return 0;
```

```
}
```



3.2.3 多分支结构

2. switch语句的格式

```
switch ( 表达式 )  
{  
  case 常量表达式1 : 语句1; [break;]  
  case 常量表达式2 : 语句2; [break;]  
    :           :           :  
  case 常量表达式n : 语句n; [break;]  
  [ default : 语句序列n+1 ]  
}
```

[...]代表可选项

注意:

- (1) switch后的表达式一般多为整型或字符型表达式
- (2) case后常量表达式的值必须互不相同
- (3) 语句1至语句n+1为多条语句时可省略{ }
- (4) 多个case可以共用一组执行语句
- (5) switch语句可以嵌套

3.2.3 多分支结构

3. switch的执行过程 (语句1--语句n后没使用break;)

(1) 计算switch后表达式的值

(2) 如果表达式的值与某个case常量表达式的值相等时, 就执行此case后的语句, 执行完后将继续执行下一个case后的语句, 如果表达式的值与所有case常量表达式的值都不同, 就执行default后的语句, 执行后结束整个switch语句

switch (表达式)

```
{ case 常量表达式1 : 语句1;  
  case 常量表达式2 : 语句2;  
    :               :  
  case 常量表达式n : 语句n;  
  default : 语句n+1;  
}
```

3.2.3 多分支结构

【例】 switch举例

```
#include<stdio.h>
```

```
int main ( )
```

```
{  int   x, y ;
```

```
    scanf("%d" , &x) ;
```

```
    switch (x)
```

```
    { case 1 : y=10 ; printf("%d\n" , y) ;
```

```
      case 2 : y=20 ; printf("%d\n" , y) ;
```

```
      case 3 : y=30 ; printf("%d\n" , y) ;
```

```
      default : y=0 ;  printf("%d\n" , y) ;
```

```
    }
```

```
    return 0;
```

```
}
```

输入: 2

输出: 20

30

0

输入: 6

输出: 0

注意: switch中
没有使用break;

3.2.3 多分支结构

◆ 执行一个case 分支后, 通常会用break结束switch语句

```
#include<stdio.h>
```

```
int main ( )
```

```
{ int x, y ;
```

```
scanf(" %d" , &x) ;
```

```
switch (x)
```

```
{ case 1 : y=10 ; printf(" %d\n" , y) ; break;
```

```
case 2 : y=20 ; printf(" %d\n" , y) ; break;
```

```
case 3 : y=30 ; printf(" %d\n" , y) ; break;
```

```
default : y=0 ; printf(" %d\n" , y) ;
```

```
}
```

```
return 0;
```

```
}
```

输入: 2

输出: 20

3.2.3 多分支结构

【例3.8】百分制成绩转换为等级制成绩，用switch实现的参考程序2

```
#include<stdio.h>
```

```
int main( )
```

```
{ int s;
```

```
scanf("%d", &s);
```

```
switch(s)
```

```
{
```

```
case s>=90 && s<=100: printf("grade is A"); break;
```

```
case s>=80 && s<90 : printf("grade is B"); break;
```

```
case s>=70 && s<80 : printf("grade is C"); break;
```

```
case s>=60 && s<70 : printf("grade is D"); break;
```

```
default: printf("grade is E");
```

```
}
```

```
return 0;
```

```
}
```

grade = $\begin{cases} A & 90 \leq s \leq 100 \\ B & 80 \leq s < 90 \\ C & 70 \leq s < 80 \\ D & 60 \leq s < 70 \\ E & 0 \leq s < 60 \end{cases}$

case后面根本不是常量表达式

3.2.3 多分支结构

【例3.8】百分制成绩转换为等级制成绩，用switch实现的参考程序3

```
#include<stdio.h>
```

```
int main( )
```

```
{ int s;
```

```
scanf("%d", &s);
```

```
switch(s/10)
```

```
{ case 10:
```

```
case 9: printf("grade is A"); break;
```

```
case 8: printf("grade is B"); break;
```

```
case 7: printf("grade is C"); break;
```

```
case 6: printf("grade is D"); break;
```

```
default: printf("grade is E");
```

```
}
```

```
return 0;
```

```
}
```

case10后没有语句，它与
case 9 共用一组执行语句



3.2.4 if的嵌套

- if语句的嵌套是指if语句的if块或else块中，又包含一个完整的if语句。嵌套的作用是对复杂条件进行判断, 实现多分支选择。
- if语句的嵌套形式

```
1. if (表达式1)
    if (表达式2) 语句;
```

```
2. if (表达式1) 语句1;
   else
       if (表达式2) 语句2;
```

```
3. if (表达式件1) 语句1;
   else
       if (表达式2) 语句2;
       else 语句3;
```

```
4. if (表达式1)
    if (表达式2) 语句1;
    else 语句2;
   else
       if (表达式3) 语句3;
       else 语句4;
```

3.2.4 if的嵌套

```
5. if① (表达式1)
    if② (表达式2) 语句1;
    else 语句2;
```

对第5种形式可以有两种理解方式:

```
(1) if① (表达式1)
    { if② (表达式2) 语句1;
      else② 语句2;
    }
```

```
(2) if① (表达式1)
    { if② (表达式2) 语句1; }
    else① 语句2;
```

C语言规定:

在if 嵌套结构中, **else**是与其前面最近的尚未配对的if 配对

3.2.4 if的嵌套

【例】 编程求解分段函数

$$y = \begin{cases} -1 & x < 0 \\ 0 & x = 0 \\ 1 & x > 0 \end{cases}$$

方法1

```
if (x < 0) y = -1;
else if (x == 0) y = 0;
else y = 1;
```

方法2

```
if (x == 0) y = 0;
else if (x < 0) y = -1;
else y = 1;
```

方法3

```
if (x >= 0)
    if (x > 0) y = 1;
    else y = 0;
else y = -1;
```

方法4

```
y = -1;
if (x != 0)
    { if (x > 0) y = 1; }
else y = 0;
```

必须加花括号，
否则出错

3.2.4 if的嵌套

【例3.9】 编程实现一个猜数游戏。程序给出一个数字，如果游戏者猜对了，显示“**You are right!**”，否则显示“**You are wrong!**”，并提示游戏者是猜大了，还是猜少了。

分析：

先判断游戏者输入的数字是否与程序给出的数字相同，

如果相同输出“**You are right!**”，结束程序；

如果不同，输出“**You are wrong!**”，然后再判断是猜大了还是猜少了。

第二个判断是嵌在第一个判断条件不成立的语句中。

3.2.4 if的嵌套

```
#include <stdio.h>
int main()
{
    int iOrigNum=555, iInputNum;    // iOrigNum表示要猜的数,
    printf("Please input a integer number: \n ");
    scanf("%d", &iInputNum);        //输入游戏者猜的数
    if (iInputNum == iOrigNum)      //判断是否相等
        printf("You are right! \n");
    else                            //else表示两个数不相等, 即猜错了
    {
        printf("You are wrong! \n");
        if (iInputNum > iOrigNum)
            printf("Your number is bigger!\n");
        else
            printf("Your number is smaller!\n");
    }
    return 0;
}
```



3.2.5 条件运算符

问：为什么会出现条件运算符？ 答：为了少写点代码呀！

有些 if 语句非常简单，可直接用条件运算来实现。

`if (a>b) max=a;
else max=b;`            `max= a>b? a : b;`

1. 条件运算符 **? :** (唯一的三目运算符)
优先级仅高于赋值类, 是右结合性
2. 条件表达式的一般形式：**表达式1 ? 表达式2 : 表达式3**
3. 执行过程：先计算表达式1，若表达式1的值为真(非0)，则计算表达式2的值, 该条件表达式的值即为表达式2 的值；若表达式1的值为假(零), 则计算表达式3 的值, 该条件表达式的值即为表达式3 的值。

3.2.5 条件运算符

$\text{max} = \text{a} > \text{b} ? \text{a} : \text{b} + 1 \longrightarrow \text{max} = (\text{a} > \text{b} ? \text{a} : \text{b} + 1)$

$\text{a} > \text{b} ? \text{a} : \text{c} > \text{d} ? \text{c} : \text{d} \longrightarrow \text{a} > \text{b} ? \text{a} : (\text{c} > \text{d} ? \text{c} : \text{d})$

注意:

- (1) 表达式1, 表达式2和表达式3 的数据类型可以不相同
- (2) 条件表达式的值的数据类型与表达式2, 表达式3中数据类型较高的那个保持一致

如 $7 > 5 ? 10 : 1.4$ 条件表达式的值为10.0

3.2.5 条件运算符

【练习3.4】输入三个整数，求最大数并输出

程序4:

```
#include<stdio.h>
```

```
int main()
```

```
{ int a, b, c, max;
```

```
scanf("%d%d%d", &a, &b, &c);
```

```
max=a;
```

```
if(b>max) max=b; →
```

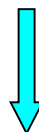
```
if(c>max) max=c;
```

```
printf("%d\n", max);
```

```
return 0;
```

```
}
```

```
if(a>b) max=a;  
else max=b;  
if(c>max) max=c;
```



```
max=a>b ? a : b ;  
max=c>max ? c : max;
```

3.2.5 条件运算符

【例3.11】 输入一个字符，如果是大写字母，转换为小写，如果不是则不转换。输出最后得到的字符。

分析：判断字符变量ch是否为大写字母，可用 `ch>='A'&&ch<='Z'` 实现，是大写需要转换，否则ch不变。大写字母转换为小写字母采用： `ch=ch+32`。

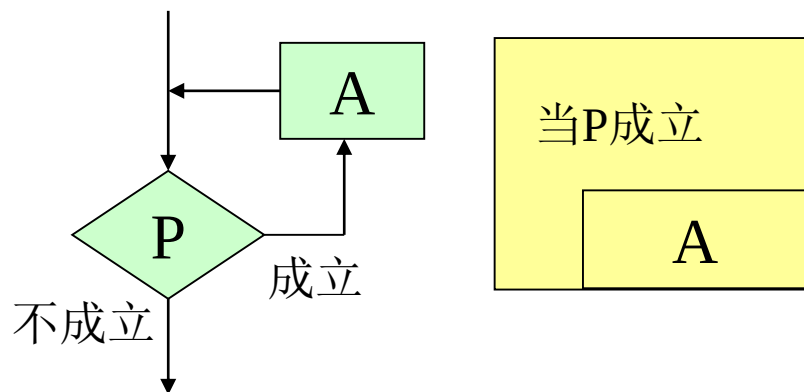
```
#include<stdio.h>
int main( )
{   char ch;
    printf("请输入一个字符： \n");
    ch=getchar();
    ch = (ch>='A'&&ch<='Z') ? (ch+32) : ch;
    printf("%c", ch);
    return 0;
}
```

3.3 循环结构

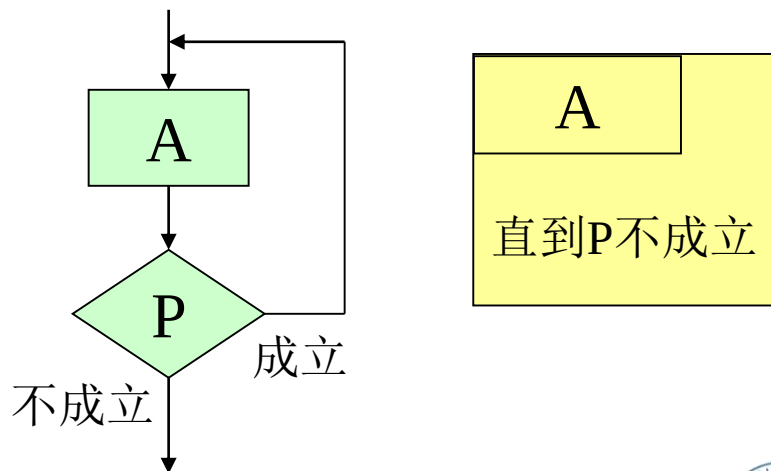
3.3.1 循环结构的引出

【例3.9】猜数游戏，只能猜一次，这样玩起来不过瘾。既然猜错时程序可以提示猜大了还猜小了，我们多猜几次应该就可以猜对了，如果要实现只有猜对了才结束程序，猜错了继续猜，程序该如何编写呢？

“当型”循环



“直到型”循环



3.3.1 循环结构的引出

➤ 请编程求解以下2个问题：

1. 请在屏幕上输出20行星号，每行星号由8个*组成。
2. 请输出1—200之间的所有偶数。

```
#include<stdio.h>
int main( )
{ printf("*****\n");
  printf("*****\n");
  printf("*****\n");
  :
  :
  printf("*****\n");
  return 0;
}
```

} 20 行

```
#include<stdio.h>
int main( )
{ printf("2, 4, 6, 8, 10,");
  printf("12,14,16,18,20,");
  printf("22,24,26,28,30,");
  :
  :
  printf("192, .....,200");
  return 0;
}
```

3.3.1 循环结构的引出

1. 什么是循环？

简单的讲，循环就是重复做一件事。

希望重复部分只写一次，
让程序控制它重复执行
20次，怎么实现呢？

【例】输出20行星号

```
#include<stdio.h>
```

```
int main( )
```

```
{
```

```
printf("*****\n");
```

```
printf("*****\n");
```

```
⋮
```

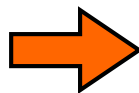
```
⋮
```

```
printf("*****\n");
```

```
return 0;
```

```
}
```

重复部分



```
#include<stdio.h>
```

```
int main( )
```

```
{ int i; // 用i作计数器
```

```
i=1;
```

```
while ( i<=20 )
```

```
{ printf("*****\n");
```

```
i=i+1;
```

```
}
```

```
return 0;
```

```
}
```

3.3.1 循环结构的引出

例8：输出1—200之间的所有偶数

```
#include<stdio.h>

int main( )
{
    printf("2, 4, 6, 8, 10,");
    printf("12,14,16,18,20,");
    printf("22,24,26,28,30,");

    printf("192, ....., 200");
    return 0;
}
```

```
#include<stdio.h>

int main( )
{ int i ;
    i=2;
    while ( i<=200 )
    { printf(" %d, ", i );
        i=i+2;
    }
    return 0;
}
```

3.3.1 循环结构的引出

2. 循环语句

C语言中的循环语句有三种：

➤ **while语句** while循环是“当型”循环, 当条件满足时执行循环体语句, 条件不满足时结束循环

➤ **do-while语句** do-while循环是“直到型”循环, 先执行循环体语句, 再判断条件是否满足, 直到条件不满足时结束循环

➤ **for语句** for循环的功能强大, 使用最为广泛
它完全可以代替while循环

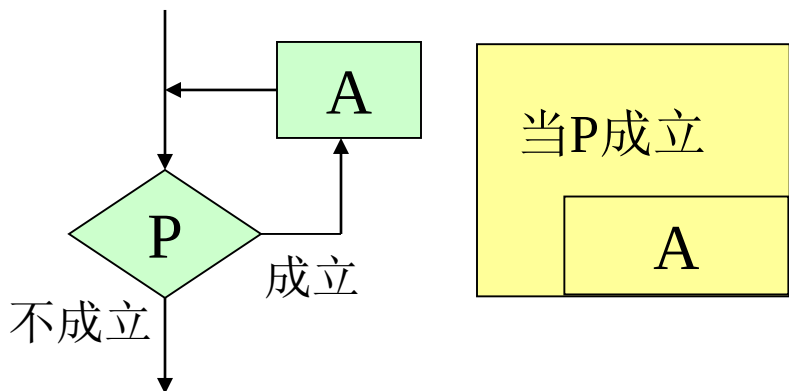
3.3.2 while循环

1. while的语法格式

while (表达式)
循环体语句;

2. 执行过程：

先计算表达式的值，
值为真时执行循环体，
值为假时结束循环



```
#include<stdio.h>
```

```
int main( )
```

```
{ int i;
```

```
  i=1;
```

表达式

```
while ( i<=20 )
```

```
{ printf(" * * * * *\n");
```

```
  i++;
```

```
}
```

循环体

```
return 0;
```

```
}
```

3.3.2 while循环

3. 使用while语句要注意四点:

- ❖ 表达式中使用的循环控制变量必须赋初值
- ❖ 表达式必须用() 括起来
- ❖ 循环体为多条语句时, 必须用{ }把它们括起来
- ❖ 循环体中必须有使循环趋向结束的语句

i=1; 循环控制变量 i 赋初值为1

while (i<=20) 表达式加小括号

{ printf(" * * * * * \n");

i++; 使循环趋向结束的语句

}

3.3.2 while循环

【例3.12】求 $1+2+3+\dots+100$ 的和

分析:

① 需要几个变量?

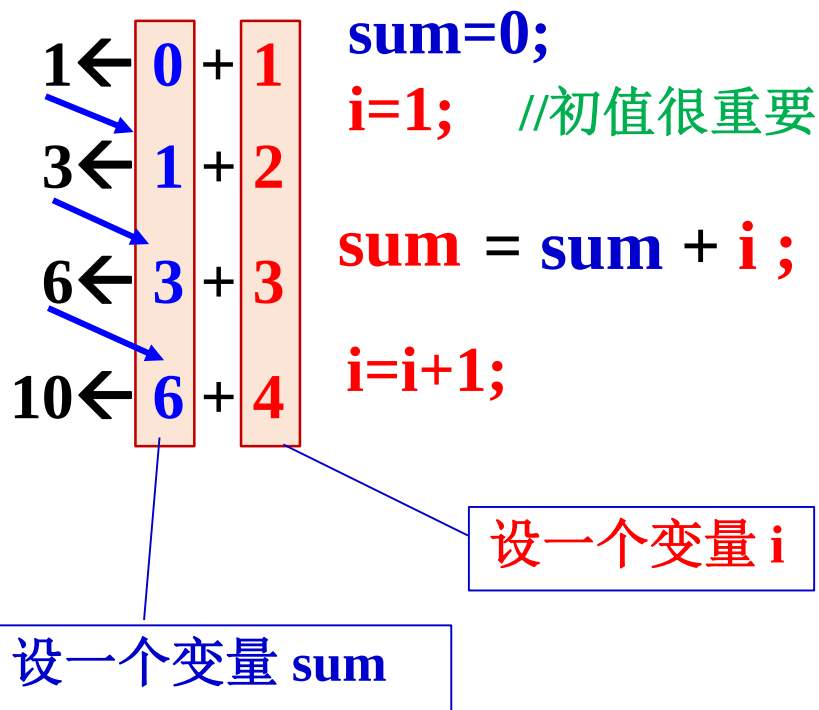
两个数相加, 还需要保存结果
3个变量大概够用吧

② 输入, 输出分别是什么?

如果自己输入 $1, 2, 3 \dots 100$,
这样也太麻烦了, 有点“low”
这些数是有规律的, 完全可以让
程序自动产生, 不需要输入
输出肯定就是最后的计算结果

③ 怎样用循环实现?

两数相加是重复做的

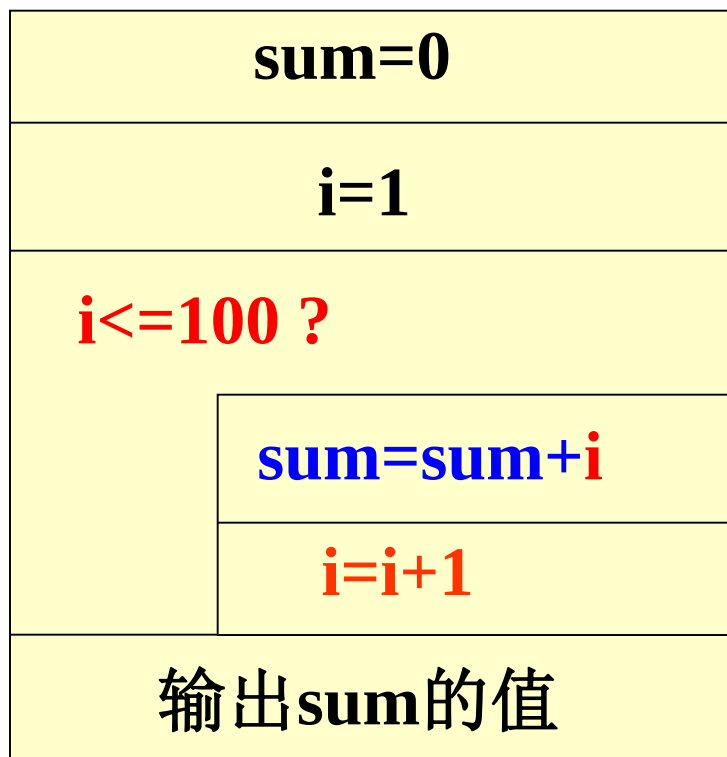


④ 是否需要一个循环控制变量?

变量 **i** 就能控制循环

3.3.2 while循环

【例3.12】的N-S图:



【例3.12】的程序代码:

```
#include <stdio.h>
```

```
int main ( )
```

```
{ int i, sum;
```

```
sum=0;
```

```
i=1;
```

```
while ( i<=100 )
```

```
{ sum=sum+i;
```

```
i++;
```

```
}
```

```
printf("sum=%d\n", sum);
```

```
return 0;
```

```
}
```

变量i用来控制循环次数, 同时又作为求和运算中的一个运算对象

3.3.2 while循环

【例】求1—n 的和, n为任意正整数, 从键盘输入

```
#include <stdio.h>
```

```
int main ( )
```

```
{ int i, sum, n ;
```

```
    sum=0;
```

```
    i=1;
```

```
    scanf("%d", &n);
```

```
    while (i<=n)
```

```
    { sum=sum+i;
```

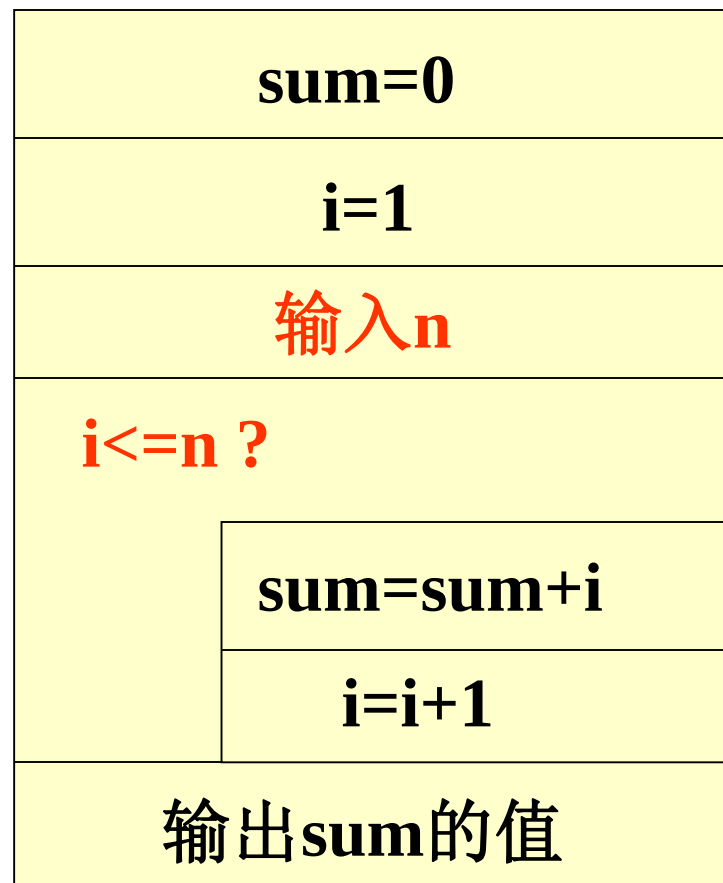
```
        i++;
```

```
    }
```

```
    printf("sum=%d\n", sum);
```

```
    return 0;
```

```
}
```



3.3.2 while循环

【例】 求某个班C语言成绩的平均分, 该班学生人数和每个学生的成绩由键盘输入

分析:

- 题目的主体结构还是累加求和, 不同之处在于累加项不是规律变化的整数, 而是学生成绩, 每个成绩都可能是不同的
- 若用 n 来表示人数, 则输入 n 就是输入学生人数
- **思考: 学生成绩需要定义几个变量?**

因为 n 个学生的成绩之和是通过 n 次循环相加得到的, 所以可以把“输入成绩”这个步骤放在循环体中, 先输入成绩, 再进行加法计算, 因此只需定义一个变量 s 来保存成绩

- 最后用 sum 除以人数 n 就可以得到平均分



3.3.2 while循环

【例】求平均分的程序代码：

```
#include <stdio.h>
```

```
int main ( )
```

```
{ int i, n ;
```

注意变量的类型

```
double s, sum, ave ;
```

```
sum=0; i=1;
```

```
scanf("%d", &n);
```

```
while (i<=n)
```

```
{ scanf("%lf", &s);
```

```
sum=sum+s ;
```

```
i++ ;
```

```
}
```

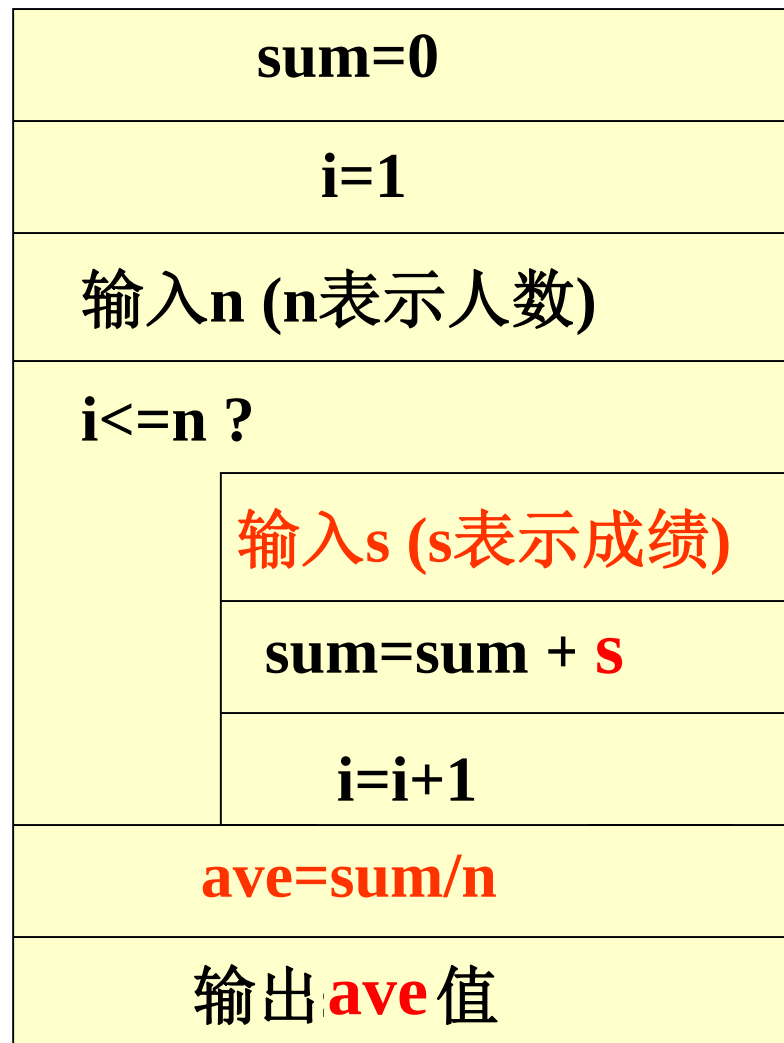
```
ave=sum/n ;
```

```
printf("ave=%6.2lf\n", ave);
```

```
return 0;
```

```
}
```

【例】求平均分的N-S图：



3.3.2 while循环

【例3.9】猜数游戏升级版，可以猜多次，直到猜对为止

```
#include <stdio.h>
```

```
int main()
```

```
{ int iOrigNum=555, iInputNum;      // iOrigNum表示要猜的数,
```

```
printf("Please input a integer number: \n ");
```

```
scanf("%d", &iInputNum);           //输入游戏者猜的数
```

```
if (iInputNum == iOrigNum)          //判断是否相等
```

```
    printf("You are right! \n");
```

```
else                                //else表示两个数不相等，即猜错了
```

```
{ printf("You are wrong! \n");
```

```
    if (iInputNum > iOrigNum)
```

```
        printf("Your number is bigger!\n");
```

```
    else printf("Your number is smaller!\n");
```

```
}
```

```
return 0;
```

```
}
```

需要重复的部分

3.3.2 while循环

【例3.9】猜数游戏升级版，可以猜多次，直到猜对为止

```
#include <stdio.h>
```

```
#include <stdlib.h> // exit函数需要包含这个头文件
```

```
int main()
```

```
{ int iOrigNum=555, iInputNum;
```

```
while( 1 ) // 不知道要猜多少次，用一个永真循环
```

```
{ printf("Please input a integer number: \n ");
```

```
scanf("%d", &iInputNum);
```

```
if (iInputNum == iOrigNum)
```

```
{ printf("You are right! \n"); exit(0); } // 猜对了用exit函数结束程序
```

```
else
```

```
{ printf("You are wrong! \n");
```

```
if (iInputNum > iOrigNum) printf("Your number is bigger!\n");
```

```
else printf("Your number is smaller!\n");
```

```
}
```

```
}
```

```
return 0;
```

```
}
```



3.3.2 while循环

【练习3.8】输入一个int型范围内的任意正整数（即该数不确定是几位数，可能是2位数，也可能是7位数），分解出它的每一位数字，并按个位、十位、...的顺序输出，测试数据如下。

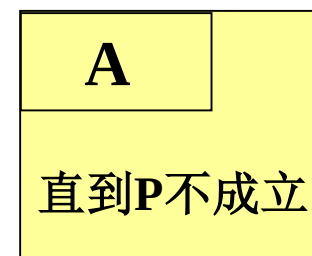
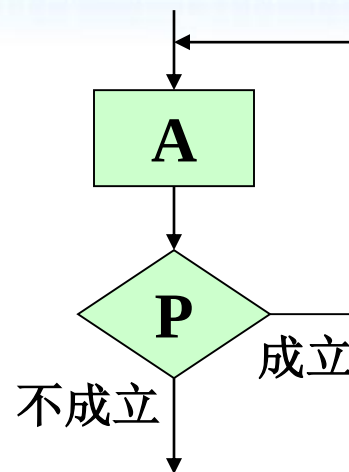
输入：34567

输出：7 6 5 4 3

3.3.3 do-while循环

1. do-while 的语法格式：

```
do  
{  
    循环体语句  
} while (表达式);
```



2. 执行过程：先执行循环体, 然后计算表达式的值, 若值为真则重复执行循环体, 若值为假则结束循环

3. 使用do-while要注意：

- ❖ 表达式中使用的循环控制变量必须赋初值
- ❖ 表达式必须用()括起来
- ❖ 循环体语句为多条语句时, 必须用{}把它们括起来
- ❖ 循环体语句中必须有使循环趋向结束的语句

3.3.3 do-while循环

【例】 用do-while语句实现求1—100 的和

sum=0	
i=1	
sum=sum+i	
i=i+1	
i<=100 ?	
输出sum的值	

```
#include <stdio.h>
int main ( )
{ int i, sum;
  sum=0;
  i=1;
  do
  { sum=sum+i;
    i++;
  } while ( i<=100) ;
  printf("sum=%d\n", sum);
  return 0;
}
```


3.3.3 do-while循环

➤ while循环和do-while循环的比较

while

- (1)先判断条件再执行循环体
- (2)循环体可能一次也不执行

```
int main( )
```

```
{ int sum=0 , i ;  
  scanf("%d", &i );
```

```
  while (i<=10)
```

```
  { sum=sum+i ;  
    i++;  
  }
```

```
  printf("%d\n", sum);  
  return 0;
```

```
}
```

若输入i值为11
则输出: 0

do-while

- 先执行循环体再判断条件
- 至少执行一次循环体

```
int main( )
```

```
{ int sum=0 , i ;  
  scanf("%d", &i );
```

```
  do
```

```
  { sum=sum+i ;  
    i++;
```

```
  } while (i<=10) ;
```

```
  printf("%d\n", sum);  
  return 0;
```

```
}
```

若输入i值为11
则输出: 11

3.3.4 for循环

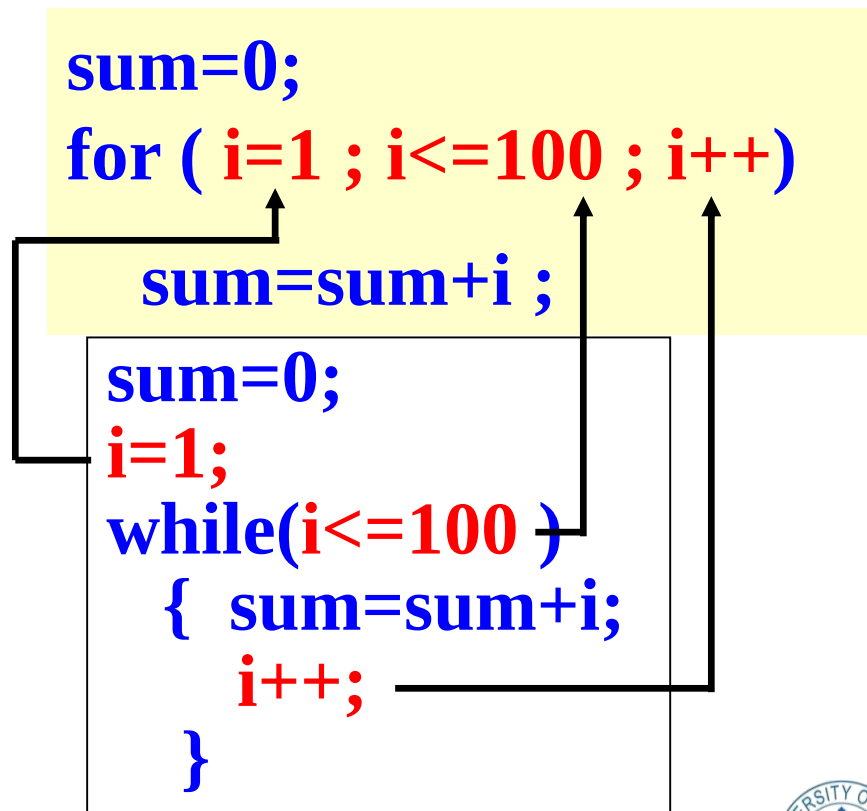
1、for 的语法格式

for (表达式1 ; 表达式2 ; 表达式3)
循环体语句;

例：将1—100求和的while循环
转换为用for循环实现

2、执行过程

- (1) 计算表达式1 (只执行一次)
- (2) 计算表达式2,
若值为真则执行循环体,
再执行第(3)步;
若值为假则结束循环
- (3) 计算表达式3,
再重复执行第(2)步



3.3.4 for循环

【例3.16】 求正整数 n 的阶乘 $n!$, n 由键盘输入

```
#include <stdio.h>
```

```
int main( )
```

```
{ int n, i;
```

```
double fact;
```

```
scanf("%d", &n);
```

```
fact=1;
```

```
for ( i=1; i<=n; i++ )
```

```
{ fact=fact*i; }
```

```
printf("% .0lf \n", fact);
```

```
return 0;
```

```
}
```

输入 n

$fact=1$

$i=1$

$i \leq n ?$

$fact=fact*i$

$i=i+1$

输出 $fact$ 的值

3.3.4 for循环

【例】用for语句实现求某个班C语言成绩的平均分

```
#include <stdio.h>
int main ( )
{ int i, n ;
  double s, sum=0, ave ;
  scanf(" %d", &n);
```

```
    for ( i=1; i<=n ; i++ )
    {   scanf(" %lf", &s);
        sum=sum+s ;
    }
```

```
    ave=sum/n ;
    printf(" %6.2lf\n", ave);
    return 0;
}
```

```
#include <stdio.h>
int main ( )
{ int i, n ;
  double s, sum=0, ave ;
  scanf(" %d", &n);
```

```
    i=1;
    while ( i<=n )
    {   scanf(" %lf", &s);
        sum=sum+s ;
        i++ ;
    }
```

```
    ave=sum/n ;
    printf(" %6.2lf\n", ave);
    return 0;
}
```



3.3.4 for循环

【例3.17】 古典问题：有一对兔子，从出生后第3个月起每个月都生一对兔子，小兔子长到第三个月后每个月又生一对兔子，假如兔子都不死，问每个月有几对兔子？

分析：兔子的规律为数列1, 1, 2, 3, 5, 8, 13, 21.....

从第三项开始，每一项都是前两项的和。

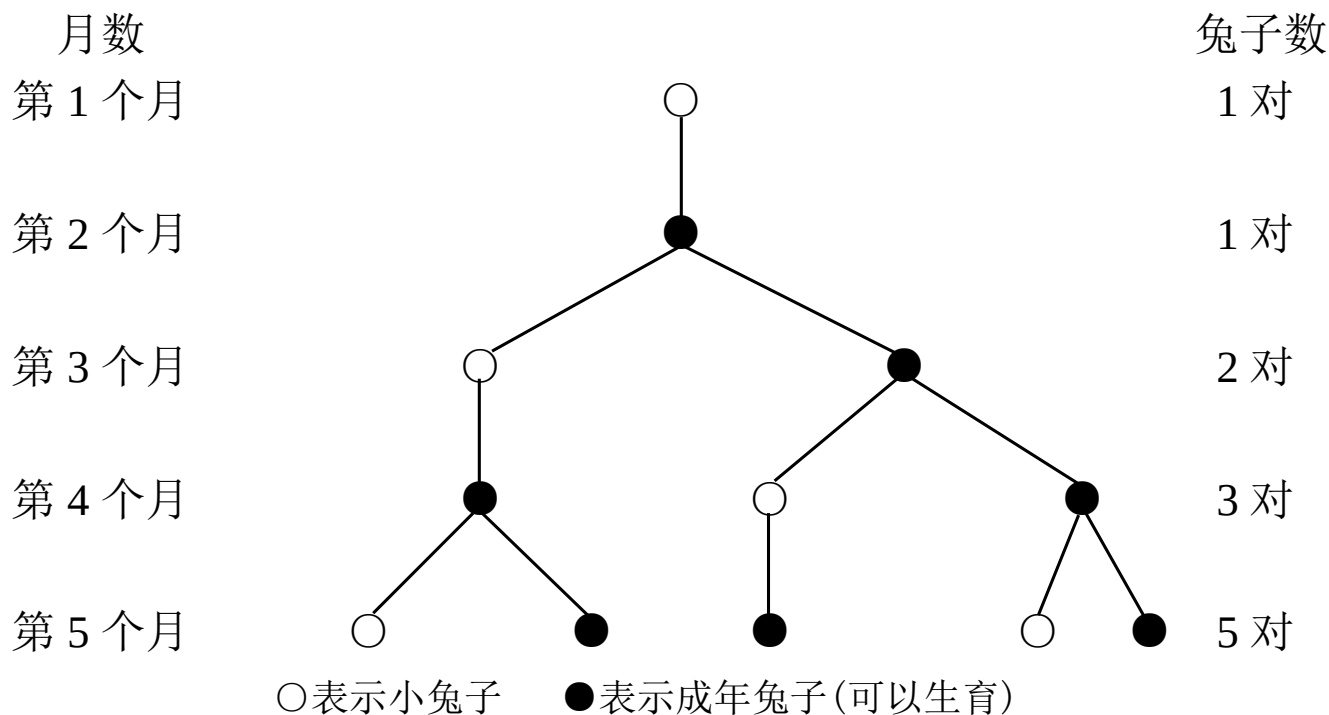


图 3.11 兔子数列

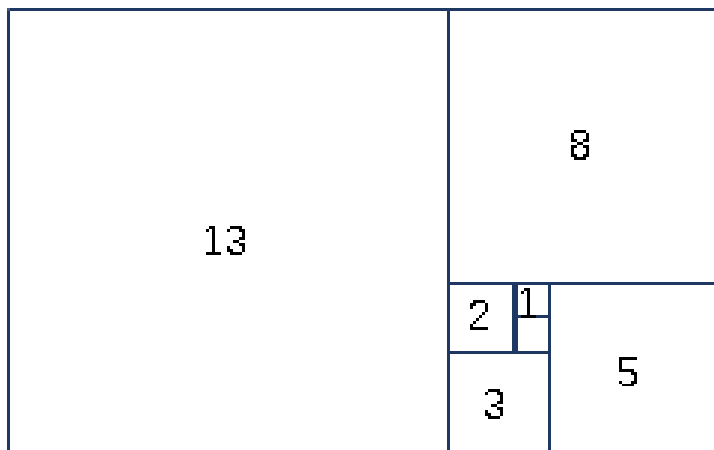
3.3.4 for循环

【例3.17】兔子序列问题

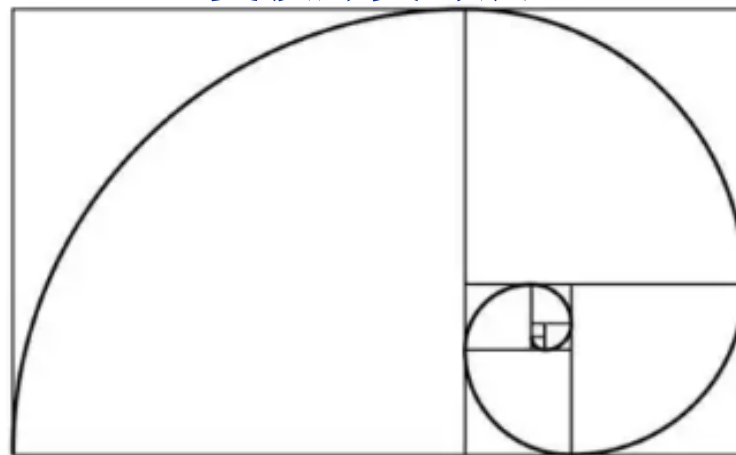
兔子数列其实就是著名的斐波那契数列(Fibonacci sequence), 又称黄金分割数列。为什么是黄金分割数列?

因为 f_n / f_{n+1} 的比值越来越逼近 0.618

斐波那契矩形



斐波那契螺旋



斐波那契数列的性质：所有小正方形的面积之和等于大矩形的面积

$$f_1^2 + f_2^2 + \cdots + f_n^2 = f_n \times f_{n+1}$$

3.3.4 for循环

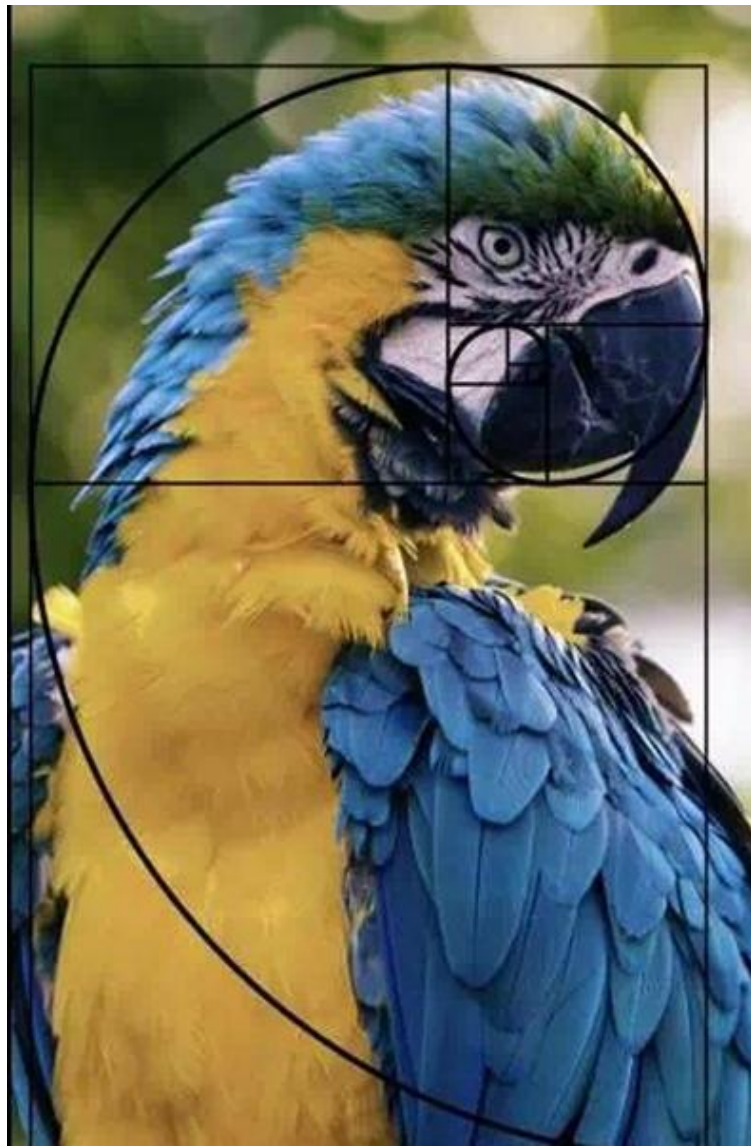
自然界的斐波那契螺旋



鸚鵡螺

3.3.4 for循环

自然界的斐波那契螺旋

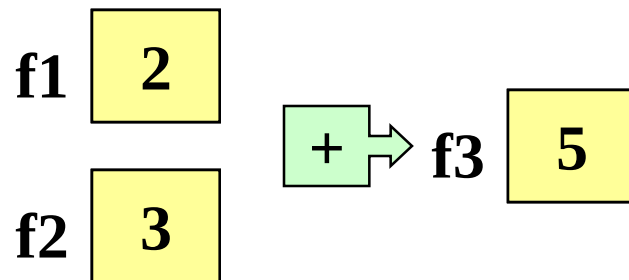


3.3.4 for循环

【例3.17】求Fibonacci数列1, 1, 2, 3, 5, 8, ...的前20个数

```
#include <stdio.h>
int main( )
{ int i, f1, f2, f3;
  f1=1;    f2=1;
  printf("%8d%8d", f1, f2);
  for ( i=3 ; i<=20 ; i++ )
  { f3=f1+f2;
    f1=f2;
    f2=f3;
    printf("%8d", f3);
    if ( i%4==0 )
      putchar('\n');
  }
  return 0;
}
```

分析数列的规律：
从第3个数开始，每个数是其前两个数之和



输出结果:

1	1	2	3
5	8	13	21
34	55	89	144
233	377	610	987
1597	2584	4181	6765

思考：分析教材P97的例3.17的程序

3.3.4 for循环

【练习】ACM1964

设计一个歌唱比赛的计分程序, 有10名专家给歌手打分, 歌手的最终成绩为去掉一个最高分和一个最低分的平均分

分析:

- 程序总体思路: 将10个成绩累加求和, 设法找出最高分和最低分, 从总分中减去最高分和最低分, 再除8得到平均分
- 如何确定最高分max和最低分min?
- max和min是否要赋初值?



3.3.5 循环嵌套

➤ 三种循环的比较:

	While	Do-while	for
循环变量初始化	写在循环之前	写在循环之前	写在表达式1
循环条件	写在while后的 () 内	写在while后的 () 内	写在表达式2
循环变量的修改	写在循环体内	写在循环体内	写在表达式3
循环体	一般不能省略	一般不能省略	可省略, 将对应操作 写在表达式2、3中
执行过程	先计算表达式 再执行循环体	先执行循环体 再计算表达式	先计算表达式1、2 再执行循环体

3.3.5 循环嵌套

- 循环的嵌套: 一个循环体内包含另一个完整的循环结构
- 按循环嵌套的层数分别称为二层循环, 三层循环...
- C语言中while, do-while, for循环可以互相嵌套

【例】 输出由 * 组成的如图所示的矩形

方法1: 无循环

```
#include <stdio.h>
int main( )
{ printf("*****\n");
  printf("*****\n");
  printf("*****\n");
  printf("*****\n");
  printf("*****\n");
  return 0;
}
```

方法2: 一层for循环

```
#include <stdio.h>
int main( )
{ int j;
  for( j=1; j<=5; j++)
    printf("*****\n");
  return 0;
}
```

```
*****
*****
*****
*****
*****
```

3.3.5 循环嵌套

方法3: 二层循环

```
#include <stdio.h>
```

```
int main( )
```

```
{ int i , j ;
```

```
  i=1;
```

```
  while (i<=5)
```

```
  { for (j=1; j<=6; j++)
```

```
    printf("*");
```

```
    printf("\n");
```

```
    i++;
```

```
  }
```

```
  return 0;
```

```
}
```

方法4: 二层for循环嵌套

```
#include <stdio.h>
```

```
int main( )
```

```
{ int i , j ;
```

```
  for( i=1; i<=5; i++)
```

```
  { for (j=1; j<=6; j++)
```

```
    printf("*");
```

```
    printf("\n");
```

```
  }
```

```
  return 0;
```

```
}
```

控制行数

控制每行的
*个数

注意: 内外层循环的控制变量不能重名
但并列循环的控制变量无此限制

3.3.5 循环嵌套

【例3.18】 输出*组成的直角三角形

```
#include <stdio.h>

int main( )
{ int i , j ;
  for( i=1; i<=5; i++)
  { for (j=1; j<=i; j++)
    putchar('*');
    putchar('\n');
  }
  return 0;
}
```

```
*
**
***
****
*****
```

特点: 第*i* 行有*i*个星号

注意内层循环的循环条件设置，因第*i*行有*i*个星号，所以这里不能写常量，而必须是变量

3.3.5 循环嵌套

【例3.19】 输出直角三角形形式的九九乘法口诀表，**每个数占4列**

```
#include <stdio.h>

int main( )
{ int i , j ;
  for( i=1; i<=9; i++)
  { for (j=1; j<=i; j++)
    printf(" %4d", i*j );
    putchar('\n');
  }
  return 0;
}
```

```
□□□1
□□□2  4
□□□3  6  9
□□□4  8 12 16
□□□5 10 15 20 25
□□□6 12 18 24 30 36
□□□7 14 21 28 35 42 49
□□□8 16 24 32 40 48 56 64
□□□9 18 27 36 45 54 63 72 81
```

3.3.5 循环嵌套

【练习3.5】 请输出下面格式的九九乘法表

```
1*1=1
1*2=2  2*2=4
1*3=3  2*3=6  3*3=9
1*4=4  2*4=8  3*4=12  4*4=16
1*5=5  2*5=10  3*5=15  4*5=20  5*5=25
1*6=6  2*6=12  3*6=18  4*6=24  5*6=30  6*6=36
1*7=7  2*7=14  3*7=21  4*7=28  5*7=35  6*7=42  7*7=49
1*8=8  2*8=16  3*8=24  4*8=32  5*8=40  6*8=48  7*8=56  8*8=64
1*9=9  2*9=18  3*9=27  4*9=36  5*9=45  6*9=54  7*9=63  8*9=72  9*9=81
```

【练习3.6】 输出如图所示星号组成的图形

```
*****
*****
***
**
*
```


3.3.5 循环嵌套

【例3.20】输出星号组成的等腰三角形。

```
#include <stdio.h>
```

```
int main( )
```

```
{ int  i, j, k ;
```

```
  for ( i=1; i<=5 ; i++)
```

```
  {  for ( j=1; j<=5-i ; j++)  
      putchar(' ');
```

```
      for ( k=1; k<=2*i-1 ; k++)  
          putchar('*');
```

```
      putchar('\n');
```

```
  }
```

```
  return 0;
```

```
}
```

特点: 第 i 行有 $2*i-1$ 个星号

若有 n 行, 则第 i 行的第 1 个星号前有 $n-i$ 个空格

```
      *  
      ***  
      *****  
      *********  
      ***********
```

控制空格的个数

控制星号的个数

3.3.5 循环嵌套

【练习3.7】输出如图所示的由星号组成的图形。

```
*****
```

```
  ****
```

```
    ***
```

```
      **
```

```
        *
```

```
      *
```

```
    **
```

```
  ***
```

```
****
```

```
*****
```

```
      *
```

```
    ***
```

```
  *****
```

```
*****
```

```
*****
```

```
    ***
```

```
      *
```

提示：
菱形可分成上下
两部分分别输出

3.3.5 循环嵌套

【例3.21】公鸡5文钱买1只，母鸡3文钱买1只，小鸡1文钱买3只，用100文钱买100只鸡，问公鸡、母鸡、小鸡各有几只？

分析：

- 假设用变量x、y、z分别代表公鸡数、母鸡数、小鸡数，根据题目描述可以列出以下两个方程：

$$x+y+z=100$$

$$5x+3y+z/3=100$$

- 由方程2可以确定x、y、z的取值范围：

$$1 \leq x < 20, \quad 1 \leq y < 33, \quad 3 \leq z < 100$$

- 为找出解答，需逐一验证x、y、z的取值，看其是否满足条件：

$$x=19, \quad y=1, \quad z=3 \xrightarrow{z \text{ 变化}} z=99;$$

$$y=2, \quad z=3 \xrightarrow{z \text{ 变化}} z=99;$$

y 变化
↓

$$y=32, \quad z=3 \xrightarrow{z \text{ 变化}} z=99;$$

➤ x、y、z这种有规律的变化可用循环嵌套实现

3.3.5 循环嵌套

【例3.21】方法1: 用三层循环实现

```
#include <stdio.h>
```

```
int main( )
```

```
{ int x, y, z;
```

```
  for (x=1; x<20; x++)
```

```
    for (y=1; y<33; y++)
```

```
      for (z=3; z<100; z+=3)
```

```
        if (5*x+3*y+z/3==100 && x+y+z==100)
```

```
          printf("x=%d, y=%d, z=%d\n", x, y, z);
```

```
  return 0;
```

```
}
```

❖ 该方法的缺点:

最内层的if语句要执行 $19*32*33$ 次

需要考虑如何减少循环执行的次数

第1层的循环体

第2层的循环体

第3层的循环体

3.3.5 循环嵌套

【例3.21】方法2: 用一层循环实现

```
#include <stdio.h>
```

```
int main( )
```

```
{ int x, y, z;
```

```
  for (z=3; z<100; z+=3)
```

```
  { x=4*z/3-100;
```

```
    if(x>0)
```

```
      { y=100-x-z;
```

```
        if ( y>0 && 5*x+3*y+z/3==100 && x+y+z==100)
```

```
          printf("x=%d, y=%d, z=%d\n", x, y, z);
```

```
      }
```

```
  }
```

```
  return 0;
```

```
}
```

$$x+y+z=100 \rightarrow y=100-x-z$$

$$5x+3y+z/3=100$$

$$\rightarrow 5x+3(100-x-z)+z/3=100$$

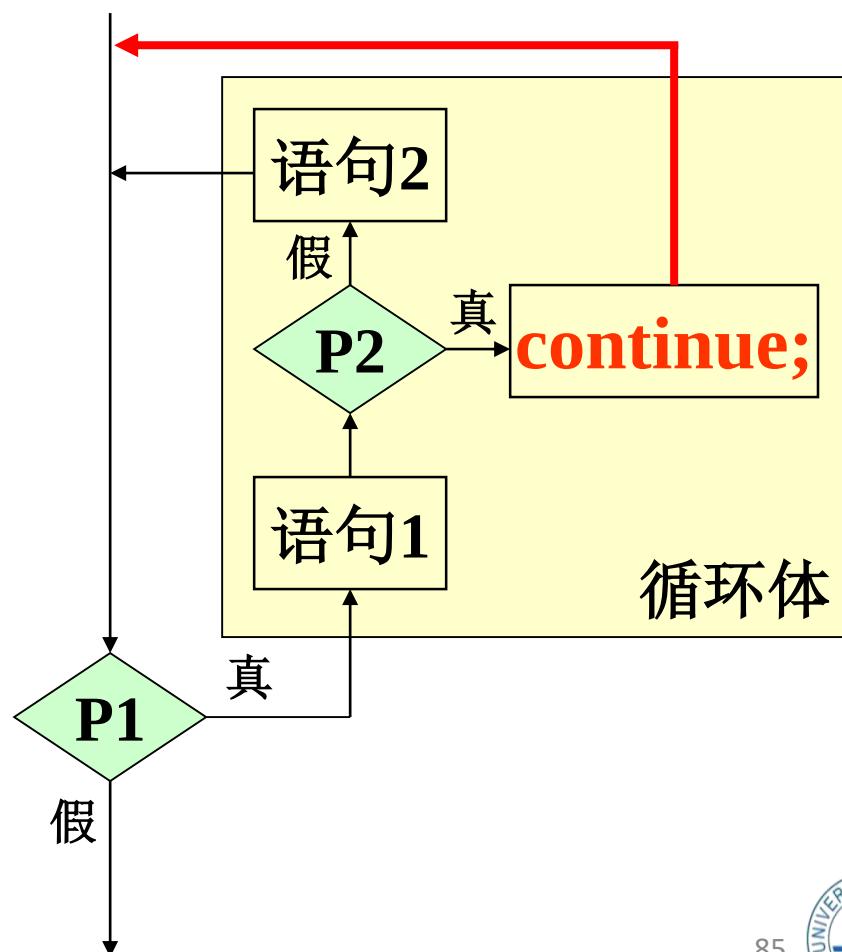
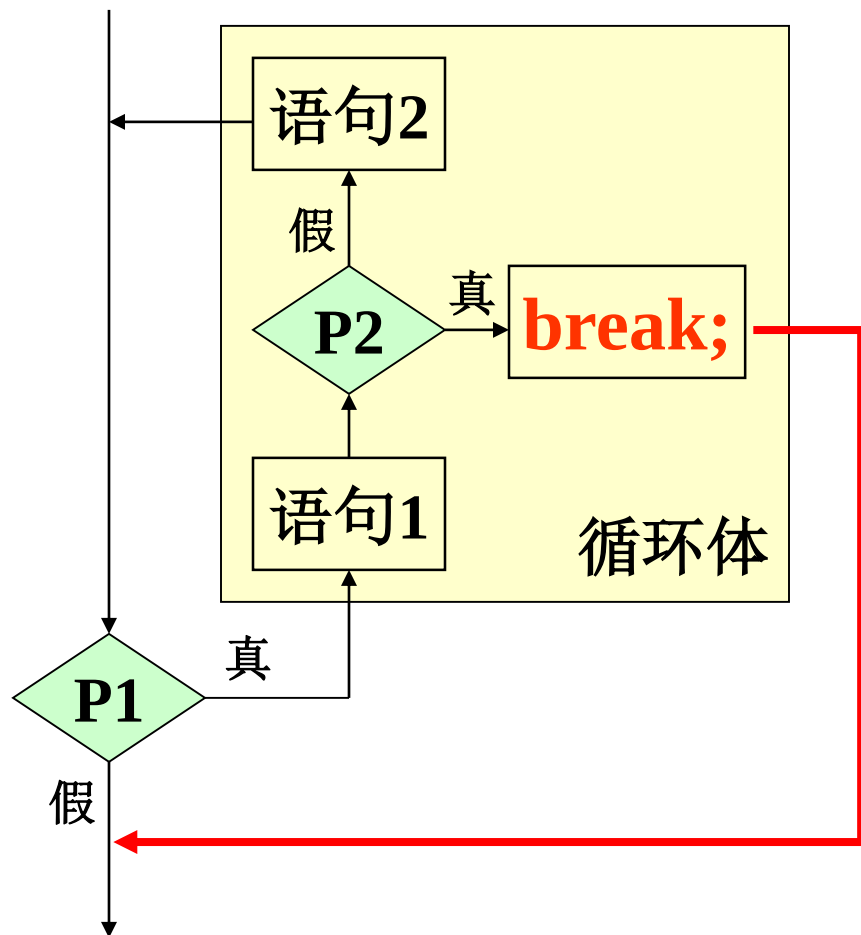
$$\rightarrow x=4z/3-100$$

求z → 求x → 求y

因此只需要z作循环变量

3.4 break和continue语句

- **break**语句：中止循环或switch结构
- **continue**语句：结束本次循环, 然后再根据是否满足条件来决定是否执行下次循环



3.4 break和continue语句

```
for(r =1 ;r <= 10 ; r++)  
{ s = 3.14*r*r ;  
  if (s>100 && s<200 )  
    break;  
  printf( "s=%6.2f\n" , s);  
}
```

输出结果:

s= 3.14

s= 12.56

s= 28.26

s= 50.24

s= 78.50

```
for(r =1 ;r <= 10 ; r++)  
{ s = 3.14*r*r ;  
  if (s>100 && s<200 )  
    continue;  
  printf( "s=%6.2f\n" , s);  
}
```

输出结果:

s= 3.14

s= 12.56

s= 28.26

s= 50.24

s= 78.50

s=200.96

s=254.34

s=314.00

3.4 break和continue语句

【例3.22】判断一个整数是否是素数。

分析：素数是一个大于1的自然数，除了1和它自身外，不能被其他自然数整除的数，素数也叫质数。

判断一个数 m 是否是素数，最直接的方法是用 m 分别去除2到 $m-1$ 间的每一个数，

如果其中有一个数能被整除，就说明 m 不是素数，

只有2到 $m-1$ 间所有的数都不能整除， m 才是素数。

如果有一个数能整除，则 m 不是素数，就不需要再继续做后面的除法，这时可以用break提前结束循环。

3.4 break和continue语句

【例3.22】判断素数参考程序1

```
#include <stdio.h>
int main( )
{   int m, i;
    scanf("%d", &m);
    for (i=2; i<m; i++)    // i<m 也可以写成 i<=m-1
    {   if (m%i==0)        // 判断m整除i的方法，是判断余数是否为0
        break;           // 能整除说明m不是素数，此时可结束for循环
    }
    if ( i == m )    // 如果i等于m，说明没有提前结束循环，m就是素数
        printf ("%d is a prime number\n", m);
    else
        printf ("%d is not a prime number\n", m);
    return 0;
}
```

分析：虽然这个方法是对的，但是效率比较差。试想一下，如果m这个数很大，比如m=98475622，循环最多会执行98475622-2=98475620次。

3.4 break和continue语句

【例3.22】判断一个整数是否是素数。

问题：如何改进判断的方法，提高效率？？？

➤ 真的有必要从2到 $m-1$ 都进行判断吗？

例如： $m=24$ ， m 除12结果为2， m 除大于12的数13, 14, ..., 23，是不是都只能得到一个1.*** 的小数，不可能再整除了，所以只需要除到 m 的一半就行了。

当 $m=98475622$ ， $98475622/2=49237811$ 次，循环次数还是很多。

➤ 有没有更快的方法呢？

这时看出数学有多重要了，数学家们证明，判断 m 是否为素数，只需要用 m 分别去除2到 $\sqrt{m}$ 就足够了。

当 $m=98475622$ ， $\sqrt{m}=9923$ ，这下效率高多了

3.4 break和continue语句

【例3.22】判断素数参考程序2—改进版1

```
#include <stdio.h>
#include<math.h>    //用平方根函数sqrt, 必须包含这个头文件
int main( )
{
    int m, i, q;
    scanf("%d", &m);
    q=sqrt((double)m);    //计算m的平方根
    for (i=2; i<=q; i++)    //注意循环条件是i<=q
    { if (m%i==0) break; }
    if ( i > q )    // 因循环条件是i<=q, 这里对应修改为 i>q
        printf ("%d is a prime number\n", m);
    else printf ("%d is not a prime number\n", m);
    return 0;
}
```

3.4 break和continue语句

【例3.22】判断素数参考程序2—改进版2

```
#include <stdio.h>
#include<math.h>
int main( )
{ int m, i, q, flag;      //加一个标志变量，用它表示是否是素数
  scanf("%d", &m) ;
  q=sqrt((double)m);
  flag=1;                  //标志变量赋值为1
  for (i=2; i<=q; i++)
  { if (m%i==0)
    { flag=0; break; }    //如果m能整除i，则标志变量改为0
  }
  if ( flag==1 )    printf ("%d is a prime number\n", m);
  else printf ("%d is not a prime number\n", m);
  return 0;
}
```



3.4 break和continue语句

【例3.23】输出100以内的所有素数

分析:

显然这个问题需要两层循环，外层循环从3~100，分别判断这个区间内的每一个奇数是否是素数。内层循环是负责判断素数的，就可以使用上面的判断方法。

```
#include <stdio.h>
#include <math.h>
int main( )
{   int m, i, q, flag;    //flag是标识变量，flag=1表示是素数
    printf("%5d", 2);    //2比较特殊，可以单独输出
    for (m=3; m<=100; m+=2) //外层循环从3~100
    {   q=sqrt((double)m);
        flag=1;
        for (i=2; i<=q; i++) //内层循环判断m是不是素数
        {   if (m%i==0)
                {   flag=0;    //flag赋值为0，表示m不是素数
                    break;    //结束内层的for循环
                }
        }
        if (flag==1) printf("%5d", m); // flag为1时输出m
    }
    return 0;
}
```



培养创新思维 提升编程能力